# JayWiki: A Cloud-Based Campus Portfolio System for Student Achievement and Professional Development

Matthew Smith
Elizabethtown College
Elizabethtown, Pennsylvania, USA
smithm6@etown.edu

## Abstract

Current methods for tracking student and alumni achievements, projects, and professional development across university campuses are fragmented, inefficient, and often non-existent. Students manage their academic projects, extracurricular activities, job experiences, and social profiles across multiple disconnected platforms, creating challenges in portfolio curation for career development and institutional assessment. This project addresses the need for a centralized, campus-integrated portfolio management system that consolidates student achievements while maintaining security and accessibility, with emphasis on student projects, coursework, awards, activities, and achievements during and after their college careers.

This paper presents JayWiki, a cloud-based portfolio management platform built on Microsoft Azure infrastructure, designed to integrate with campus authentication systems while supporting external user access. The system enables students to document academic projects, professional experiences, social profiles such as GitHub and LinkedIn, event participation, and awards within a unified interface. The motivation stems from the growing importance of comprehensive digital portfolios in modern career development and the need for institutions to track student outcomes effectively.

The primary contribution of this work is a scalable, enterprise-grade architecture that leverages direct OAuth integration via `angular-oauth2-oidc` and ASP.NET Core for multi-provider authentication, supporting Microsoft Entra ID (via the common endpoint, accommodating both organizational and personal Microsoft accounts), Google OAuth 2.0, and local email/password authentication. The system architecture combines Angular with TypeScript and Tailwind CSS for responsive frontend interfaces, ASP.NET Core Web API with Entity Framework Core for backend services, Azure SQL Database for relational data management, and Azure Blob Storage for multimedia content. The implementation demonstrates automated deployment workflows using GitHub Actions, Azure App Service, and Azure Static Web Apps.

The resulting platform supports complex student data relationships including course enrollments, project ownership and collaboration, job history, and event participation, backed by a 14-entity relational schema deployed to Azure SQL Database. The system demonstrates secure multi-provider authentication via a policy-based JWT scheme selector, intuitive interfaces for portfolio creation and management, and a faculty-managed course catalog enabling structured academic project association. The platform serves as a viable cloud-native solution for educational portfolio management, designed for post-graduation handoff to the Elizabethtown College School of Engineering and Computer Science.

## CCS Concepts

• **Applied computing** → **E-learning**; • **Information systems** → *Web applications*; Cloud computing; • **Security and privacy** → *Authentication.*

## Keywords

ePortfolio, cloud computing, student portfolio management, Azure, authentication, single sign-on, educational technology, Angular, ASP.NET Core, digital portfolios

## 1 Introduction

In today's competitive job market, students require comprehensive digital portfolios that effectively showcase their academic achievements, professional experiences, and extracurricular involvement [3]. Traditional portfolio management approaches rely on fragmented systems where students maintain separate records across multiple platforms like GitHub for code repositories, LinkedIn for professional networking, personal websites for project showcases, and university systems for academic records [1]. This fragmentation creates inefficiencies in portfolio curation and limits institutional visibility into student outcomes and achievements.

### 1.1 Existing Problems

The current landscape of student portfolio management presents several critical challenges. First, data fragmentation forces students to manually synchronize information across multiple platforms, leading to inconsistent or outdated portfolio representations. Second, lack of institutional integration prevents universities from effectively tracking student achievements, project outcomes, and career development metrics in a centralized manner. Third, authentication complexity emerges when students graduate or external stakeholders (alumni, employers, collaborators) need access to portfolio content but lack campus credentials [4]. Finally, limited multimedia support in existing academic systems restricts students from showcasing project demonstrations, event photos, and rich media content that effectively communicate their achievements.

### 1.2 Existing Solutions and Shortcomings

Several solutions currently address aspects of student portfolio management. LinkedIn provides professional networking and experience tracking but lacks academic project depth and institutional integration. Students cannot effectively showcase detailed technical projects, event participation, or campus-specific achievements within LinkedIn's professional networking framework. GitHub excels at code repository management but offers limited support for non-technical projects, event documentation, or comprehensive

professional profiles. University Learning Management Systems (LMS) such as Canvas or Blackboard maintain academic records but typically lack career development features, external accessibility for alumni, and multimedia project showcasing capabilities [7].

These existing solutions share three critical shortcomings. First, single-purpose design limits their effectiveness as each platform serves one function well but fails to provide holistic portfolio management [3]. Second, lack of flexible authentication prevents seamless access across campus users, alumni, and external collaborators; most systems require either campus credentials (excluding alumni) or public access (compromising security) [5]. Third, minimal cloud-native architecture in traditional academic systems results in poor scalability, limited mobile access, and inadequate support for modern deployment practices [2].

### 1.3 Proposed Solution

This project introduces JayWiki, a cloud-based Campus Portfolio Management System for the Elizabethtown College School of Engineering and Computer Science that addresses these shortcomings through three distinctive design aspects.

First, a multi-provider authentication architecture using direct OAuth integration via `angular-oauth2-oidc` and ASP.NET Core supports three independent identity providers: Microsoft Entra ID via the common endpoint (supporting both organizational `@etown.edu` accounts and personal Microsoft accounts), Google OAuth 2.0, and local email/password authentication backed by BCrypt password hashing and backend-issued JWTs. This eliminates the authentication barrier that typically excludes graduated students or external collaborators, without requiring campus tenant administrator access.

Second, a unified data model consolidates academic projects, professional experiences, social profiles, event participation, and multimedia content within a 14-entity relational schema, allowing students to manage their complete portfolio from a single interface while maintaining referential integrity across different achievement types. A faculty-managed course catalog enables structured association between student enrollments and academic projects.

Third, Microsoft ecosystem alignment leverages Azure infrastructure to support institutional IT policies, combining Azure Blob Storage for public-facing multimedia hosting, Azure SQL Database for relational data management, Azure App Service for backend hosting, and Azure Static Web Apps for the Angular frontend. This architecture provides a foundation for post-graduation handoff to the department and potential future integration with campus systems via Microsoft Graph API.

The backend utilizes ASP.NET Core Web API with Entity Framework Core for type-safe database operations and RESTful service design. The frontend employs Angular 17 with TypeScript and Tailwind CSS for responsive, mobile-friendly interfaces. Automated deployment through GitHub Actions ensures continuous integration and delivery across both hosting tiers.

### 1.4 Results Overview

The implemented platform delivers a fully functional portfolio management system supporting complex student data relationships including course enrollments, project ownership and collaboration, job history, and event participation. Authentication across all three providers was validated end-to-end, with a policy-based JWT scheme selector routing tokens to the correct validation scheme based on the `iss` claim. The system supports role-based access control distinguishing students, instructors, and administrators, with instructor-restricted course catalog management and owner/collaborator-scoped project editing. Automated CI/CD pipelines deploy both frontend and backend on every push to the main branch, and the platform is designed for maintainability and handoff with comprehensive documentation covering architecture, schema, authentication flows, setup procedures, and lessons learned.

### 1.5 Paper Organization

The remainder of this paper is organized as follows. Section 2 provides detailed analysis of existing portfolio management solutions and related work in educational technology platforms. Section 3 presents the architectural framework, database schema based on entity-relationship modeling, and authentication flow diagrams. Section 4 describes the completed implementation of the system across all components. Section 5 describes the testing methodology and evaluation of the implemented system. Section 6 outlines the eight-week development timeline and deliverables. Finally, Section 7 summarizes contributions and directions for future work.

## 2 Background

The development of digital portfolio systems for educational institutions has been an active area of research and commercial development for over two decades. This section examines three significant contributions to student portfolio management, analyzing their achievements and identifying areas where improvements remain necessary.

### 2.1 Mahara ePortfolio System: Open-Source Portfolio Management

The Mahara Project, led by researchers at multiple international universities including Danube University Krems (Austria), Dublin City University (Ireland), and Monash University (Australia), developed Mahara as an open-source ePortfolio platform beginning in the mid-2000s [4].

Mahara established itself as one of the most widely adopted open-source portfolio systems in higher education. The platform successfully implemented granular access control allowing students to share specific portfolio artifacts with selected audiences, Learning Tools Interoperability (LTI) integration enabling seamless connection with learning management systems like Moodle, and collaborative group spaces for peer feedback and discussion forums [1]. Research evaluations demonstrated positive student acceptance, with studies showing favorable ratings for usefulness and future intention to use [1]. The system's architecture supports reflective learning practices by enabling students to contextualize evidence, align work with competency standards, and create

narrative portfolios suitable for academic assessment, professional development, and employment applications [3].

Despite widespread adoption, Mahara implementations have revealed significant usability challenges. Case studies documented that the system's feature depth creates a complex conceptual model for students, resulting in complicated processes even for creating simple portfolio views [4]. The interface requires substantial upfront training and technical support, with institutions reporting that dedicated helpdesks are essential for successful implementation [5]. Authentication limitations present another barrier as Mahara typically relies on institutional single sign-on systems, creating access problems for alumni once students graduate [5]. Storage scalability issues also emerge as multimedia portfolios grow, with centralized server architectures struggling under the load imposed by video, audio, and high-resolution image content [2]. Finally, limited support for modern identity providers prevents flexible authentication models that could support both campus users and external stakeholders [1].

## 2.2 Cloud-Based Portfolio Systems: Scalability and Infrastructure Research

Researchers at various universities investigated cloud computing architectures for ePortfolio systems, publishing findings in the *Handbook of Research on Cloud-Based STEM Education for Improved Learning Outcomes* [2, 6].

This research stream identified fundamental scalability problems in traditional ePortfolio implementations and proposed cloud-based solutions to address storage and performance limitations [2]. The work demonstrated that cloud computing platforms could solve storage constraints plaguing centralized server architectures, enabling multimedia-rich portfolios without imposing artificial storage limits [6]. Researchers showed that cloud-based systems could provide 24/7 accessibility from any internet-connected device, supporting the mobile-first expectations of modern students. The studies also established that cloud infrastructure could support data-driven institutional assessment, providing analytics for accreditation processes and program evaluation [2].

The research revealed three critical challenges in cloud-based portfolio adoption. First, authentication and identity management remain problematic as most proposed systems assumed simple username/password or basic institutional SSO, failing to address multi-provider authentication supporting current students, alumni, and external collaborators simultaneously [6]. Second, data portability and ownership concerns created adoption barriers, with students and institutions expressing anxiety about vendor lock-in [3]. Third, institutional resistance emerged as a significant barrier, with IT departments concerned about data sovereignty, privacy compliance with FERPA and institutional data protection policies, and the security implications of storing student work on external cloud platforms [2].

## 2.3 Commercial Learning Platforms: Anthology Portfolio and Enterprise Solutions

Major educational technology vendors including Anthology (formerly Blackboard), Digication, and Creatrix Campus developed commercial portfolio systems integrated with comprehensive learning management platforms, targeting institutional-scale deployments [7, 9].

Commercial systems achieved significant integration depth with existing educational technology ecosystems. These platforms implemented enterprise-grade security features including role-based access control, encryption at rest and in transit, automated backups, and compliance with educational data protection regulations [9]. The systems demonstrated institutional scalability, successfully supporting tens of thousands of concurrent users across large university systems [7, 8].

Commercial systems face three fundamental limitations that restrict their applicability to modern educational environments. First, inflexible authentication models typically support only institutional SSO or proprietary account systems, failing to accommodate hybrid authentication needs for campuses serving both traditional students and non-traditional populations such as alumni and external partners [1, 5]. This limitation particularly affects alumni engagement, as graduated students lose portfolio access once institutional credentials expire. Second, platform lock-in and limited interoperability create significant concerns as proprietary data formats and limited export capabilities make migration between systems difficult, increasing long-term costs [3, 8]. Third, one-size-fits-all design means commercial systems optimize for generic use cases rather than specialized institutional needs, making customization difficult without expensive consulting engagements [2, 9].

## 2.4 Addressing Identified Gaps: This Project's Approach

This project directly addresses the limitations identified in existing portfolio systems through three architectural innovations that distinguish it from prior work.

First, the platform implements a hybrid multi-provider authentication architecture using direct OAuth integration via angular-oauth2-oidc and ASP.NET Core JWT Bearer authentication. Unlike Mahara's institutional-only SSO or commercial platforms' proprietary account systems, this approach supports three independent identity providers: Microsoft Entra ID via the common endpoint (accommodating both organizational `@etown.edu` accounts and personal Microsoft accounts without requiring campus tenant administrator access), Google OAuth 2.0, and local email/password authentication backed by BCrypt password hashing and backend-issued JWTs. This eliminates the alumni access barrier documented in [5] while maintaining campus security requirements through industry-standard OAuth 2.0 protocols. The authentication implementation avoids third-party identity services (Auth0, Azure AD B2C) that require special IT permissions or introduce vendor dependencies, instead leveraging direct OAuth provider integration through well-established client libraries.

Second, the system adopts a cloud-native architecture on Microsoft Azure infrastructure, addressing the scalability and storage limitations identified in [2, 6]. The deployment leverages Azure App Service B1 tier for consistent backend performance without cold-start delays, Azure SQL Database for relational data management with Basic tier (5 DTUs) providing adequate performance for academic workloads, and Azure Blob Storage for multimedia content

with planned CDN integration. Unlike proprietary commercial systems, this architecture uses open standards including RESTful APIs, OAuth 2.0, standard SQL, and well-documented Azure services, enabling institutional IT staff to maintain, customize, and migrate the system without the vendor lock-in concerns raised in [3, 8]. The estimated monthly operational cost of approximately $18.50 remains well within student Azure credit allocations, demonstrating cost-effective cloud deployment for educational institutions.

Third, the platform integrates with Microsoft campus ecosystems rather than competing with existing infrastructure. By aligning with campus Microsoft environments through Entra ID integration and Azure-based deployment, the system reduces the institutional resistance documented in [2]. The technology stack—Angular with TypeScript and Tailwind CSS for frontend development, ASP.NET Core Web API with Entity Framework Core for backend services—provides modern development practices while maintaining compatibility with enterprise IT standards. The architecture provides a foundation for future enhancements such as Microsoft Graph API integration for automated data synchronization with campus Microsoft 365 systems, addressing the institutional integration gaps identified in existing solutions [1, 7]. This campus-aligned approach combined with modern web technologies delivers the usability improvements needed to address Mahara's complexity issues documented in [4] while maintaining enterprise-grade security and scalability through automated deployment via GitHub Actions to Azure App Service and Azure Static Web Apps.

The database schema design addresses data fragmentation by consolidating academic projects (linked to courses), professional experiences (jobs and social profiles), event participation, awards, and multimedia content within a unified relational model using Entity Framework Core migrations. This unified approach contrasts with existing systems that typically handle only one aspect of student achievement (e.g., LinkedIn for professional profiles, GitHub for code repositories) or require manual synchronization across platforms. The implementation demonstrates that modern cloud platforms, when properly architected with hybrid authentication and campus ecosystem integration, can overcome the scalability, accessibility, and institutional adoption barriers that have limited previous ePortfolio systems.

## 3 System Design

This section presents the architectural design of the campus portfolio management system, detailing the system architecture, core components, and technology stack decisions. The design emphasizes scalability, security, and integration with campus Microsoft ecosystems while maintaining accessibility for alumni and external users.

### 3.1 System Architecture

The platform employs a three-tier cloud-native architecture deployed on Microsoft Azure infrastructure, as illustrated in Figure 1. The presentation tier consists of an Angular single-page application (SPA) hosted on Azure Static Web Apps, providing responsive user interfaces accessible from desktop and mobile browsers. The application tier implements RESTful services through an ASP.NET Core Web API deployed on Azure App Service (B1 tier), handling business logic, authentication validation, and data processing. The data tier utilizes Azure SQL Database (Basic tier, 5 DTUs) for relational data management and Azure Blob Storage for multimedia content including project images, videos, and event photos.

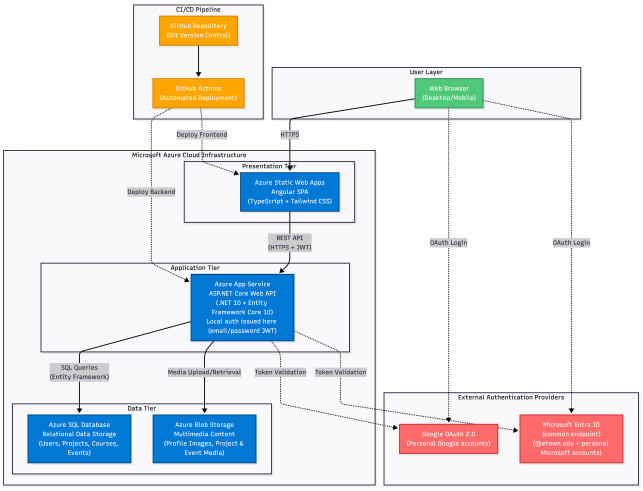


**Figure 1: System architecture showing three-tier cloud deployment on Azure infrastructure. The frontend Angular SPA communicates with the ASP.NET Core API via HTTPS, which connects to Azure SQL Database for structured data and Azure Blob Storage for multimedia content.**

User authentication follows OAuth 2.0 protocol flows, supporting three distinct identity providers as shown in Figure 2: Microsoft Entra ID via the common endpoint (supporting both organizational `@etown.edu` and personal Microsoft accounts without requiring campus tenant administrator access), Google OAuth 2.0, and local email/password authentication backed by BCrypt password hashing and backend-issued JWTs signed with HMAC-SHA256. The frontend implements OAuth flows using the `angular-oauth2-oidc` library for the two external providers, while local authentication is handled directly via backend API endpoints. The backend validates JWT tokens using a triple authentication scheme in ASP.NET Core, with a policy-based scheme selector that inspects the token `iss` claim to route to the appropriate validation scheme (Google, Microsoft, or Local). This multi-provider architecture eliminates the alumni access barrier present in traditional campus systems while maintaining enterprise security standards.

The deployment architecture leverages GitHub Actions for continuous integration and deployment (CI/CD). Code commits to the main branch trigger automated workflows that build the Angular frontend, compile the .NET backend, and deploy both components to Azure. The Static Web Apps deployment workflow handles frontend build optimization including TypeScript compilation, Tailwind CSS processing, and asset bundling. The App Service deployment workflow executes backend compilation, runs Entity Framework migrations against the production database, and deploys the compiled API to Azure App Service.

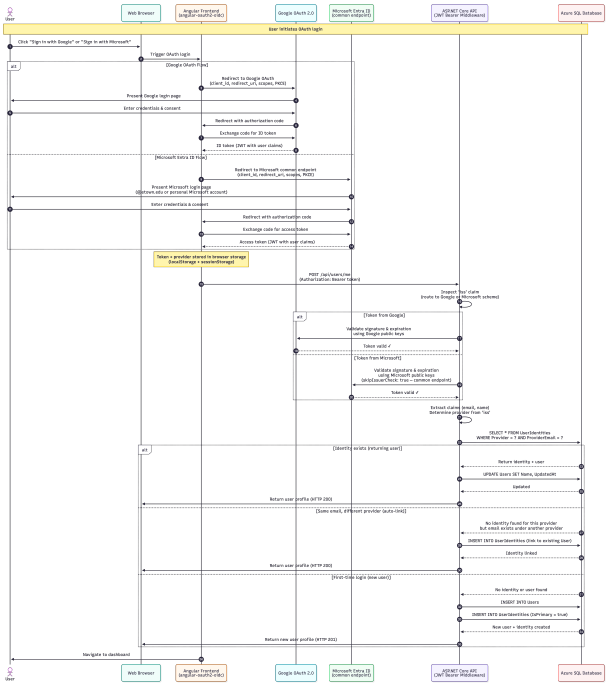


**Figure 2: Authentication flow diagram showing OAuth 2.0 flows for Microsoft Entra ID (common endpoint, supporting `@etown.edu` and personal Microsoft accounts), Google OAuth 2.0, and local email/password authentication. The `angular-oauth2-oidc` library handles frontend token acquisition for OAuth providers, while ASP.NET Core JWT Bearer middleware with a policy-based triple-scheme selector validates all tokens on the backend.**

## 3.2 Core Components

*3.2.1 Backend API Services.* The backend implements a RESTful API architecture organized into seven primary controller modules, each handling a specific domain area of the application:

- **UsersController:** Manages user profile operations including initial user provisioning on first OAuth login, profile retrieval and updates, profile image upload to Azure Blob Storage, and linked identity management.
- **AuthController:** Handles local email/password authentication via `POST /api/auth/register`, `POST /api/auth/login`, and `POST /api/auth/link` for connecting a local identity to an existing OAuth account.
- **CoursesController:** Manages the faculty-controlled course catalog (`COURSE_CATALOG`) with instructor/admin-restricted create, update, and delete operations, and handles student course enrollment records (`COURSE`).
- **ProjectsController:** Handles CRUD operations for academic projects with nested endpoints for topic tagging, media management, and collaborator assignment.
- **EventsController:** Provides endpoints for campus events and registrations, implementing event creation, participant registration, and media management functionality.
- **JobsController:** Manages professional experience records including employment history with companies, titles, and date ranges.
- **SocialsController:** Manages external social profile links including GitHub, LinkedIn, and personal websites.

All API endpoints enforce authentication via the `[Authorize]` attribute with policy-based authorization, requiring valid JWT tokens from any of the three supported identity providers. The backend implements comprehensive error handling with standardized HTTP status codes (200 for success, 400 for validation errors, 401 for authentication failures, 404 for missing resources, 500 for server errors) and structured JSON error responses. Cross-Origin Resource Sharing (CORS) policies allow requests from the Azure Static Web Apps frontend domain while blocking unauthorized origins.

Entity Framework Core handles all database interactions through a code-first approach. The `ApplicationDbContext` class defines entity configurations, relationships, and constraints using Fluent API syntax. Database migrations track schema changes over time, with the migration history establishing the fourteen core entities and their relationships. The ORM provides type-safe LINQ queries that compile to optimized SQL, automatic change tracking for update operations, and support for complex queries including joins, filtering, and pagination.

*3.2.2 Database Schema.* The relational database schema implements fourteen entities with carefully designed relationships to support comprehensive portfolio management, as illustrated in the Entity-Relationship Diagram (Figure 3). The schema centers on the `USER` entity, which serves as the primary hub connecting all other data domains.

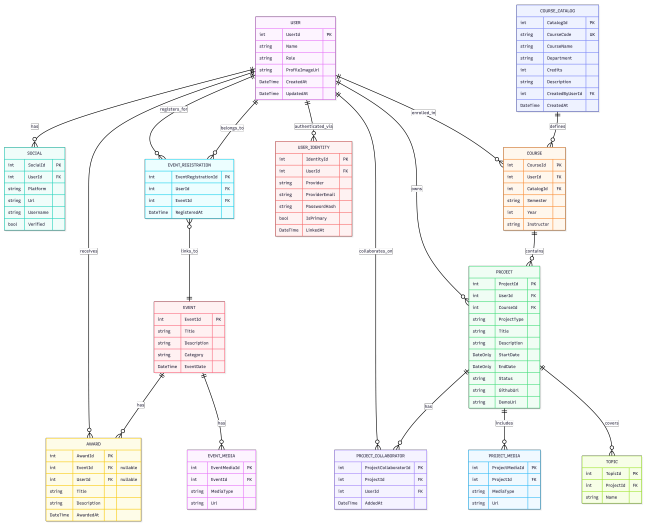


**Figure 3: Entity-Relationship Diagram showing the complete database schema. The USER entity serves as the central hub, connecting to identity data (USER_IDENTITY), academic data (COURSE_CATALOG, COURSE, PROJECT), professional data (JOB, SOCIAL), and campus involvement (EVENT_REGISTRATION, AWARD) through well-defined relationships.**

The USER table stores core profile information including name, role (student, instructor, or admin), profile image URL, and timestamp fields. Authentication identity is decoupled into the USER_IDENTITY table, which stores one row per linked provider per user, each containing the provider name (google, microsoft, or local), the provider-scoped email address, and a BCrypt password hash for local accounts. A composite unique constraint on (Provider, ProviderEmail) prevents duplicate identities per provider. This decoupled model allows a single user to link multiple authentication providers to one account, with all user lookups scoped by both provider and email to prevent cross-provider identity confusion.

Academic achievement tracking implements a hierarchical structure anchored by a faculty-managed COURSE_CATALOG entity that stores canonical course codes, names, and department information, editable only by instructors and administrators. Students enroll in catalog entries via the COURSE entity (storing semester, year, and instructor), and courses contain projects via the PROJECT entity (including titles, descriptions, dates, GitHub URLs, and demo URLs). Projects associate with topics via the TOPIC entity for categorization and filtering, and with collaborators via the PROJECT_COLLABORATOR junction table. Projects also link to multimedia content through PROJECT_MEDIA, supporting images, videos, and external links that showcase project demonstrations and documentation. Projects may be standalone (no course association) or linked to a specific course enrollment.

Professional development features utilize JOB entities to track employment history with companies, titles, date ranges (with nullable end dates indicating current positions), and descriptions. The SOCIAL entity manages external profile links including GitHub, LinkedIn, personal websites, and other platforms, with a verified flag indicating whether the platform has confirmed ownership.

Campus involvement tracking leverages a many-to-many relationship between users and events implemented through the EVENT_REGISTRATION junction table. The EVENT entity categorizes activities (clubs, sports, academic events, other) and stores event metadata including titles, descriptions, and dates. Events connect to EVENT_MEDIA for photos and videos, and to AWARD entities that capture recognition received through event participation.

The schema implements cascade delete behavior selectively. Deleting a user removes associated jobs, socials, courses, and event registrations. Deleting a project removes its topics, media, and collaborator entries. Deleting an event removes registrations, media, and awards. Notably, PROJECT.UserId and PROJECT.CourseId are both configured with DeleteBehavior.NoAction in Entity Framework Core to avoid SQL Server's multiple cascade path restriction; project cleanup on user or course deletion is handled at the application layer instead. Foreign key indexes on all relationship columns optimize join performance for common queries.

*3.2.3 Frontend Architecture.* The Angular frontend implements a modular component-based architecture using standalone components. The authentication module handles OAuth flows, token management, and protected route guards using angular-oauth2-oidc. The portfolio module contains components for displaying user profiles, project listings, and experience timelines. The courses module manages course catalog browsing and student enrollment views. The events module manages event browsing, registration, and participation tracking.

Shared services handle cross-cutting concerns: the AuthService manages authentication state and provider-aware token retrieval, and an HTTP interceptor automatically attaches the correct Bearer token per provider on every outgoing API request. Component styling utilizes Tailwind CSS utility classes for responsive layouts that adapt seamlessly between desktop, tablet, and mobile viewports. TypeScript interfaces define strong contracts between frontend and backend, with models matching the C# entity classes to ensure type safety across the full stack.

State management follows reactive patterns using RxJS. Form handling utilizes Angular's reactive forms with built-in validation, providing immediate feedback on invalid inputs before submission.

## 3.3 Technology Stack

The technology selections reflect three strategic priorities. First, Microsoft ecosystem alignment throughout the stack (ASP.NET Core, Azure services, Microsoft Entra ID) facilitates campus IT approval and future institutional maintenance after project handoff. Second, type safety spanning frontend TypeScript, backend C#, and database-entity mappings via Entity Framework reduces runtime errors and improves developer productivity. Third, cloud-native architecture with managed services (App Service, SQL Database, Static Web Apps) minimizes operational overhead while providing enterprise-grade reliability and security suitable for production campus deployment.

Alternative technologies were considered and rejected with specific justifications. Node.js backend frameworks were evaluated but dismissed due to weaker campus IT alignment and preference for C# type safety. React was considered for the frontend but Angular's superior enterprise form handling and Microsoft ecosystem fit made it preferable. PostgreSQL and MongoDB were evaluated as database alternatives but Azure SQL's tighter Microsoft integration and Entity Framework optimization led to its selection. Auth0 and Azure AD B2C were initially considered for authentication but rejected due to third-party dependencies (Auth0) and IT permission barriers requiring tenant administrator access (Azure AD B2C), leading to the direct OAuth integration approach using angular-oauth2-oidc.

The deployment utilizes an Azure for Students subscription providing $100 credit, with monthly operational costs estimated at $18.50 (App Service B1 $13, SQL Database Basic $5, Blob Storage $0.50), enabling five months of continuous development and testing within the available budget. This cost structure demonstrates that cloud-native educational applications can be developed and operated cost-effectively while maintaining production-grade infrastructure and security standards.

# 4 Implementation

This section describes the completed implementation of the JayWiki campus portfolio management system, covering the database schema, backend API, authentication infrastructure, frontend application, cloud deployment, and CI/CD pipeline.

**Table 1: Technology Stack Components and Selection Rationale**

| Technology | Purpose | Selection Rationale |
|---|---|---|
| Angular 17+ | Frontend framework for SPA development | Enterprise-grade form handling, Microsoft ecosystem alignment, TypeScript integration, prior project experience |
| TypeScript | Strongly-typed JavaScript for frontend and backend (C#) | Compile-time type checking, superior IDE support, improved maintainability for institutional handoff |
| Tailwind CSS v3.4.1 | Utility-first CSS framework for responsive UI | Rapid development without custom CSS, design consistency, responsive layout support; v4 incompatible with Angular 17 build system |
| ASP.NET Core (.NET 10) | Backend web framework for REST API | Native Microsoft stack, excellent Azure integration, enterprise performance, comprehensive security features |
| Entity Framework Core | ORM for database operations and migrations | Type-safe queries, automated schema management, relationship handling, LINQ syntax |
| Azure SQL Database | Cloud relational database (Basic tier, 5 DTUs) | Microsoft integration, automated backups, encryption at rest/transit, adequate academic workload performance |
| Azure Blob Storage | Object storage for multimedia content | Scalable, low cost, CDN integration support, public-access blob model suitable for portfolio media |
| Azure App Service | Backend hosting platform (B1 tier) | Managed infrastructure, no cold starts (unlike free F1 tier), consistent dedicated compute for demo and shared use |
| Azure Static Web Apps | Frontend hosting with global CDN | Free SSL certificates, global distribution, edge caching, GitHub Actions integration |
| angular-oauth2-oidc | OAuth 2.0/OIDC client library for Angular | Lightweight, actively maintained, comprehensive OAuth support, avoids third-party auth services |
| BCrypt.Net-Next | Password hashing library for local accounts | Industry-standard BCrypt algorithm; passwords never stored in plain text |
| GitHub Actions | CI/CD automation platform | Tight GitHub integration, free tier, workflow-as-code, built-in Azure deployment actions |
| Git + GitHub | Version control and collaboration | Industry standard, project requirement, branch-per-feature workflow with pull requests |

## 4.1 Database Schema and Migrations

The complete relational database schema was implemented using Entity Framework Core with a code-first migration approach. Fourteen entities are defined in C# model classes with Fluent API configurations: USER, USER_IDENTITY, JOB, SOCIAL, COURSE_CATALOG, COURSE, PROJECT, PROJECT_COLLABORATOR, PROJECT_MEDIA, TOPIC, EVENT, EVENT_REGISTRATION, EVENT_MEDIA, and AWARD. Four migrations track the full schema evolution: `InitialCreate`, `AddProjectCollaborators`, `AddCourseCatalogAndUserRole`, and `AddUserIdentities`.

The `USER` table stores name, role (`student`, `instructor`, `admin`), profile image URL, and timestamps. Authentication identity is decoupled into `USER_IDENTITY`, which stores one row per linked provider per user, each containing the provider name (`google`, `microsoft`, or `local`), provider-scoped email, and BCrypt password hash for local accounts. A composite unique constraint on (Provider, ProviderEmail) prevents duplicate identities per provider, and all user lookups are scoped by both fields to prevent cross-provider identity confusion.

Key schema constraints include: a composite unique index on event registrations preventing duplicate entries per user, a composite unique index on project collaborators preventing duplicate assignments, a unique constraint on course codes (stored uppercase), and `DeleteBehavior.NoAction` on both project-to-user and project-to-course foreign keys to avoid SQL Server's multiple cascade path restriction, with application-layer cleanup handling project deletion instead.

## 4.2 Backend API

The ASP.NET Core (.NET 10) Web API is fully implemented and deployed, organized into seven controller modules:

- **UsersController:** User provisioning on first OAuth login, profile retrieval and updates, profile image upload to Azure Blob Storage (5 MB max, JPEG/PNG/GIF/WEBP), and linked identity management including primary identity designation and provider unlinking.
- **AuthController:** Local email/password authentication via register, login (BCrypt verification with static dummy hash to prevent timing-based enumeration), and an account-linking endpoint for connecting a local identity to an existing OAuth account.
- **CoursesController:** Faculty-managed course catalog with instructor/admin-restricted create, update, and delete operations, public catalog browsing with department and search filters, and student enrollment record management.
- **ProjectsController:** Full CRUD for academic projects with nested endpoints for topic management, media management, and collaborator assignment. Supports status and type query filters. Projects may be standalone or associated with a course enrollment. Ownership is verified via the project

owner field; collaborators are checked via the collaborator join table.
- **EventsController:** Event creation, participant registration, and media management with duplicate registration prevention via composite unique index.
- **JobsController:** Employment history CRUD with nullable end dates for current positions.
- **SocialsController:** External social profile link management including GitHub, LinkedIn, and personal websites.

A shared base controller provides reusable authorization helpers for provider-scoped user lookup, project owner checks, project membership checks, and instructor/admin role validation. Standardized HTTP status codes are used throughout (200, 201, 400, 401, 403, 404, 409, 500). CORS policies allow requests from the Azure Static Web Apps frontend domain.

Azure Blob Storage integration supports profile image upload with host validation, an allowlisted container set, concurrent container caching, and upload-before-delete ordering to ensure the old blob is only removed after the new URL is safely persisted.

## 4.3 Authentication Infrastructure

Three fully operational authentication providers are supported end-to-end.

**Google OAuth 2.0:** The frontend uses `angular-oauth2-oidc` with PKCE code flow to acquire a Google ID token. After OAuth completes, the session restore handler calls the user provisioning endpoint, which creates or updates the user and identity rows. The backend validates Google ID tokens against the configured client ID audience.

**Microsoft Entra ID:** The app registration uses the common endpoint, registered under a personal `etownjaywiki@outlook.com` account (school IT blocks tenant-level registration). This supports both organizational `@etown.edu` accounts and personal Microsoft accounts. Issuer checking is disabled on both frontend and backend because tokens from the common endpoint carry tenant-specific issuers that do not match the common endpoint URL. Security is maintained via cryptographic signature verification and audience validation.

**Local Email/Password:** The backend issues HMAC-SHA256 signed JWTs (issuer: `jaywiki-api`, 7-day expiry) after BCrypt password verification. A static dummy hash computed at class initialization prevents timing-based user enumeration on failed lookups. Local tokens are stored in `localStorage` under a distinct key, separate from the OAuth library's own storage keys.

The backend implements a policy-based JWT scheme selector that inspects the `iss` claim to route each incoming token to the correct validation scheme (Google, Microsoft, or Local). Tokens from unrecognized issuers are rejected before any database access. A race condition on concurrent first logins is handled by catching SQL unique constraint violation error codes on save and re-querying for the concurrently created user.

## 4.4 Frontend Application

The Angular 17 frontend is fully implemented using standalone components, Tailwind CSS v3.4.1, and TypeScript. The following pages are complete:

- **Dashboard / Profile:** Dual-mode view supporting own profile editing and public read-only view of other users' profiles. Displays projects, courses, jobs, and social links.
- **Projects (public):** Public gallery of all ECS student projects with full-text search across title, description, topics, and student name, plus type/status filter dropdowns and a clear-filters control.
- **Explore:** Browsable student directory.
- **MyCourses / CourseDetail:** Student course enrollment management linked to the faculty-managed course catalog.
- **MyProjects:** Project management page with a modal component for create and edit operations. All HTTP state is owned by the parent page and passed to the modal via Angular input and output bindings.
- **ProjectDetail:** Shared component rendering in editable mode for owners/collaborators and read-only mode for public visitors.
- **Public Courses Catalog:** Browsable view of the faculty-managed course catalog with department and search filters.

An HTTP interceptor reads the active provider from session storage to determine which token to attach: OAuth tokens via `angular-oauth2-oidc` for Google and Microsoft, and a locally stored token for local accounts. The session restore function is called exactly once in the app initializer to avoid race conditions on OAuth code exchange.

## 4.5 Cloud Deployment and CI/CD

The complete Azure infrastructure is deployed within the `campus-portfolio-rg` resource group:

- **Azure SQL Server:** `jaywiki-server-ms` (West US 2) with firewall rules permitting Azure service access.
- **Azure SQL Database:** Basic tier (5 DTUs, West US 2).
- **Azure App Service:** `jaywiki-api-ms`, Linux B1 tier (East US), hosting the ASP.NET Core API. HTTPS redirection middleware is omitted as Azure handles SSL termination at the load balancer. All secrets are stored as App Service Application Settings.
- **Azure Static Web Apps:** Hosting the Angular frontend with global CDN and free SSL. Build output is configured to the browser subdirectory reflecting Angular 17's updated builder output path.
- **Azure Blob Storage:** Profile images and project media stored in public-access blob containers. Unique blob naming prevents collisions.

GitHub Actions CI/CD workflows deploy both tiers automatically on every push to the main branch. The frontend workflow handles TypeScript compilation, Tailwind CSS processing, and asset bundling. The backend workflow compiles the API and runs Entity Framework migrations against the production database. Monthly operational costs of approximately $18.50 (App Service B1 $13, SQL Database Basic $5, Blob Storage $0.50) remain within the $100 Azure for Students credit allocation.

# 5 Evaluation

This section describes the testing and evaluation performed on the JayWiki campus portfolio management system, covering API validation, authentication verification, frontend usability, and security authorization testing.

## 5.1 API Endpoint Testing

All backend API endpoints were validated using Postman against both the local development environment and the production Azure deployment. Each controller module was tested systematically across its full set of operations:

- **UsersController:** User provisioning via `POST /api/users/me` was verified to correctly create `USER` and `USER_IDENTITY` rows on first login for all three providers, and to update existing records on subsequent logins. Profile image upload was tested with valid JPEG and PNG files, verifying correct storage in Azure Blob Storage and URL persistence in the database. File size and type validation was confirmed to reject files exceeding 5 MB and unsupported formats.
- **AuthController:** Local registration was verified to create a BCrypt hashed identity and return a signed JWT. Login was confirmed to reject incorrect passwords with HTTP 401 and accept correct credentials with a valid token. The link endpoint was tested to successfully attach a local identity to an existing OAuth account.
- **CoursesController:** Instructor-only operations (create, update, delete catalog entries) were confirmed to return HTTP 403 when called with a student-role token. Public catalog browsing and department/search filters were validated for correct results. Deletion was verified to be blocked when active enrollments exist.
- **ProjectsController:** Project creation, retrieval, update, and deletion were tested for both standalone and course-associated projects. The `?status=` and `?type=` query filters were validated to return correctly filtered result sets. Collaborator add and remove operations were confirmed to enforce owner-only access. Topic and media nested endpoints were verified for correct foreign key association.
- **EventsController, JobsController, SocialsController:** Standard CRUD operations were validated for correct HTTP status codes, response shapes, and ownership enforcement.

Standardized HTTP status codes were confirmed throughout: 200 and 201 for successful reads and creates, 400 for validation errors, 401 for missing or invalid tokens, 403 for insufficient role permissions, 404 for missing resources, and 409 for constraint violations such as duplicate enrollments or collaborators.

## 5.2 Authentication Validation

All three authentication providers were validated end-to-end in both local and production environments.

**Google OAuth 2.0:** The complete PKCE code flow was exercised from login button click through token acquisition, `POST /api/users/me` call, and dashboard navigation. Token attachment via the HTTP interceptor was confirmed for subsequent authenticated requests.

**Microsoft Entra ID:** Login was tested with both an organizational `@etown.edu` account and a personal Microsoft account via the common endpoint, confirming that both account types are supported. The `skipIssuerCheck: true` configuration was verified to resolve the tenant-specific issuer mismatch that otherwise causes token rejection.

**Local Email/Password:** Registration, login, and the link flow were tested in sequence. The `local_token` localStorage key was confirmed to be read correctly by the HTTP interceptor when `sessionStorage auth_provider` is set to `"local"`, without conflicting with `angular-oauth2-oidc`'s own storage keys.

The policy-based `MultiScheme` JWT selector was validated by submitting tokens from each provider to protected endpoints, confirming correct routing based on the `iss` claim. Tokens with unrecognized issuers were confirmed to be rejected with HTTP 401 before any database access occurs.

## 5.3 Authorization Testing

Manual authorization testing verified role-based and ownership-based access control across all protected endpoints:

- Requests submitted without a Bearer token returned HTTP 401 across all protected endpoints.
- Requests submitted with an expired or malformed token returned HTTP 401, confirming JWT validation rejects invalid tokens before reaching controller logic.
- Student-role tokens attempting instructor/admin-restricted operations (course catalog management) returned HTTP 403.
- Authenticated users attempting to modify another user's projects, courses, or profile data returned HTTP 403, confirming ownership checks via `IsProjectOwnerAsync()` and `GetCurrentUserAsync()` function correctly.
- Collaborator-role access was tested to permit project edits while blocking delete and collaborator management operations, matching the intended authorization model.

## 5.4 Frontend and Integration Testing

The Angular frontend was evaluated through browser-based testing in both local development (`localhost:4200`) and the production Azure Static Web Apps deployment. All implemented pages were exercised for correct data loading, form submission, and navigation behavior:

- The Projects public page was tested for search and filter functionality, confirming that queries across title, description, topics, and student name return accurate results and that the clear-filters control resets state correctly.
- The MyProjects page and `ProjectFormModal` were tested for project create and edit workflows, verifying that modal state is correctly initialized for both new and existing projects.
- The ProjectDetail component was confirmed to render in editable mode for owners and collaborators and in read-only mode for unauthenticated or unaffiliated visitors.
- Profile image upload was tested end-to-end, confirming that the new image appears immediately after upload and that the previous blob is removed from Azure Blob Storage.

- The CI/CD pipeline was validated by pushing changes to the main branch and confirming that both the frontend and backend GitHub Actions workflows completed successfully and that the updated versions were live in production.

The production deployment confirmed that removing the HTTPS redirection middleware resolves the redirect loop caused by Azure's SSL termination at the load balancer, and that the Angular build output path is correctly configured in the Static Web Apps deployment workflow.

# 6 Development Timeline

Development spanned eight weeks organized into focused phases that built incrementally toward a fully deployed and evaluated platform.

The mid-April 2026 academic deadline was met with all core deliverables complete: a fully deployed three-tier Azure architecture, triple-provider authentication, a fourteen-entity relational schema, seven API controller modules, and a complete Angular frontend. The project is designed for post-graduation handoff to the Elizabethtown College School of Engineering and Computer Science, with comprehensive documentation supporting future maintainers.

# 7 Conclusion and Future Work

## 7.1 Conclusion

This paper presented JayWiki, a cloud-based campus portfolio management system designed for the Elizabethtown College School of Engineering and Computer Science. The platform addresses the fragmentation of existing portfolio tools by consolidating academic projects, course enrollments, professional experiences, social profiles, event participation, and awards within a unified, publicly accessible interface.

The core contributions of this work are: a multi-provider authentication architecture supporting Microsoft Entra ID (common endpoint), Google OAuth 2.0, and local email/password authentication via a policy-based triple-scheme JWT selector; a fourteen-entity relational schema deployed to Azure SQL Database with a decoupled `USER_IDENTITY` model supporting multiple linked providers per account; a faculty-managed course catalog enabling structured association between course enrollments and student projects; and a complete Angular 17 frontend with a public projects gallery, dual-mode profile views, and collaborative project editing. The system is fully deployed on Microsoft Azure infrastructure with automated CI/CD via GitHub Actions, and is documented for post-graduation institutional handoff.

The implementation demonstrates that a small-team capstone project can produce a production-grade, cloud-native educational platform within a single academic semester and within the $100 Azure for Students credit allocation, while addressing authentication barriers that have historically excluded alumni from institutional portfolio systems.

## 7.2 Future Work

Several enhancements are identified for future development following institutional handoff.

**Table 2: Development Timeline**

| Week | Focus and Deliverables |
|---|---|
| 1 | Foundation and environment setup: GitHub repository, Azure resource provisioning (App Service, SQL Database, Blob Storage), Angular and ASP.NET Core project initialization, local development environment configuration. |
| 2 | Database schema and infrastructure: all fourteen entity model classes defined, initial Entity Framework Core migration executed against Azure SQL Database, firewall rules and App Service configuration finalized. |
| 3 | Core backend APIs and authentication architecture: controller scaffolding, OAuth provider registrations (Google Cloud Console, Microsoft Entra ID), JWT Bearer middleware configuration, core CRUD endpoints for USER, COURSE, PROJECT, and TOPIC. |
| 4 | Authentication completion and API finalization: end-to-end OAuth flows for Google and Microsoft, local email/password auth via `AuthController`, triple-scheme JWT selector, user provisioning via `POST /api/users/me`, `[Authorize]` enforcement, Azure Blob Storage integration. |
| 5 | Frontend components and integration: Angular standalone components for Dashboard, MyProjects, ProjectDetail, MyCourses, and CourseDetail; `ProjectFormModal` extraction; HTTP interceptor for provider-aware token attachment; RxJS reactive data patterns. |
| 6 | Feature completion and poster: Projects public page with search and filter, public Courses catalog, Explore page, events subsystem, jobs and socials management, `COURSE_CATALOG` instructor controls, poster finalized for academic presentation. |
| 7 | Testing and evaluation: Postman API validation across all controllers, end-to-end authentication testing for all three providers, manual authorization testing for role and ownership enforcement, browser-based frontend and integration testing in local and production environments. |
| 8 | Bug fixes, CI/CD finalization, and final presentation: GitHub Actions workflows validated for both frontend and backend, production deployment stabilized, handoff documentation completed (`LESSONS_LEARNED.md`, `DB_SCHEMA.md`, `AUTH_FLOW.md`, `ARCHITECTURE.md`, `projectsetup.md`), final presentation delivered. |

**Unit and Integration Testing:** No automated test suite was implemented during the initial development period. Future maintainers should prioritize unit tests for `AuthService` and the JWT scheme selector, integration tests for the user provisioning flow, and end-to-end tests covering core portfolio workflows. A testing framework such as xUnit for the backend and Jasmine/Karma for the Angular frontend would be appropriate starting points.

**Project Visibility Controls:** The current schema exposes all projects publicly. Adding a `Visibility` field (`public`, `unlisted`, `private`) to the `PROJECT` entity would allow students to control which work is externally visible, which is particularly relevant for in-progress or sensitive projects.

**Role Assignment Interface:** Instructor and admin roles are currently assigned manually via direct database edits. A role management UI accessible to administrators would allow the department to promote users without requiring database access.

**Account Merging:** Students who authenticate with multiple providers independently before linking their accounts currently end up with separate user records. An account merge endpoint would allow these to be consolidated without data loss.

**Microsoft Graph API Integration:** The Azure-based architecture provides a natural foundation for future integration with Microsoft 365 via Graph API, enabling automated synchronization of course enrollments from campus systems, calendar integration for events, and Teams-based notifications.

**Full-Text Search:** The current search on the Projects page performs in-memory filtering. Adding full-text search indexes on `PROJECT.Title` and `PROJECT.Description` in Azure SQL Database would improve performance as the dataset grows.

**Resource Co-location:** The current deployment places the App Service in East US and the SQL Database in West US 2, introducing approximately 60–80ms of cross-region latency per database query. Co-locating all resources within a single Azure region would reduce this to under 5ms and eliminate cross-region data transfer costs.

**Analytics Dashboard:** A Power BI embedded dashboard surfacing aggregate metrics — projects per semester, technology topic frequency, event participation rates, and alumni engagement — would support departmental assessment and accreditation reporting, which was identified as a gap in existing ePortfolio systems.

## Artifact Availability

The JayWiki source code is available at https://github.com/Etown-CS/JayWiki. A live demonstration of the deployed system is accessible at https://www.youtube.com/watch?v=g_37lvpSnxg.

## References


[1] I. Balaban and G. Bubas. 2010. Educational potentials of ePortfolio systems: Student evaluations of Mahara and Elgg. In *Proceedings of the 32nd International Conference on Information Technology Interfaces (ITI 2010)*. IEEE, Cavtat, Croatia, 329–336. https://doi.org/10.1109/ITI.2010.5546436

[2] P. Gupta. 2016. The emergence of cloud portfolio in higher education. In L. Chao (Ed.), *Handbook of Research on Cloud-Based STEM Education for Improved Learning Outcomes*. IGI Global, Hershey, PA, USA, 31–40. https://doi.org/10.4018/978-1-4666-9924-3.ch003

[3] L. Nguyen and B. Fussell. 2016. Evaluating technologies for education: Case of ePortfolio. *Technology, Innovation and Education* 2, 1 (2016), Article 5, 1–17. https://doi.org/10.1186/s40660-016-0010-8

[4] I. Buchem, G. Attwell, and R. Torres. 2011. The Mahara ePortfolio system: Implementation and use at the Nelson Marlborough Institute of Technology. In *Proceedings of the International Conference on ePortfolio*. European Institute for E-Learning, Maastricht, Netherlands.

[5] C. Shepherd and D. Bolliger. 2011. Mahara ePortfolio implementation: Challenges and lessons learned. *Australasian Journal of Educational Technology* 27, 7 (2011), 1129–1143.

[6] S. Thanuskodi. 2016. Cloud computing applications in education. In M. S. Khine (Ed.), *Handbook of Research on Cloud-Based STEM Education for Improved Learning Outcomes*. IGI Global, Hershey, PA, USA, 1–18. https://doi.org/10.4018/978-1-4666-9954-9.ch001

[7] H. Coates, R. James, and G. Baldwin. 2005. A typology for learning management systems: How Blackboard is used in higher education. *Higher Education Research & Development* 24, 1 (2005), 19–34. https://doi.org/10.1080/0729436052000318568

[8] B. Brandon and A. Hollingshead. 2005. Blackboard as a learning management system: A review of its effectiveness. *Journal of Educational Technology Systems* 33, 1 (2005), 5–15.

[9] Anthology Inc. 2022. *Anthology Portfolio: Enabling Authentic Assessment and Lifelong Learning*. Anthology Inc. https://www.anthology.com